

Funções, Tratamento de Exceções e Bibliotecas em Python

1. O Poder das Funções

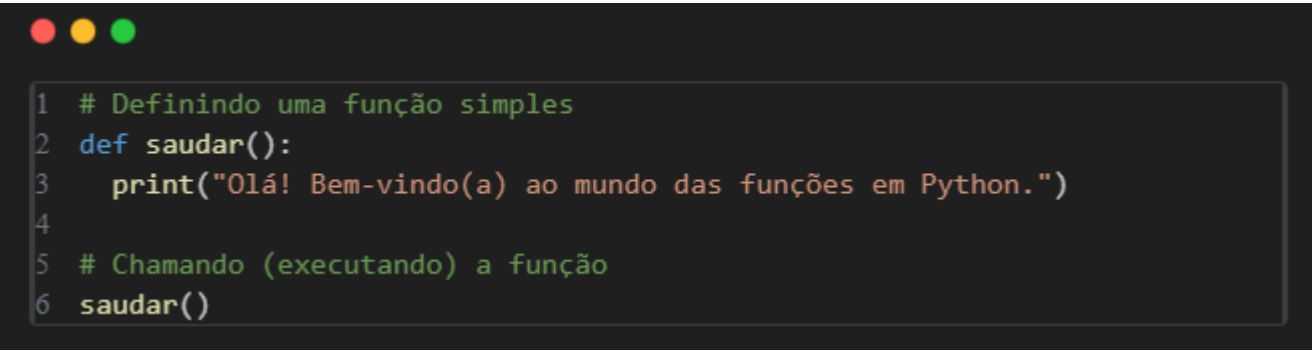
Funções são blocos de código reutilizáveis que realizam uma tarefa específica. Elas são um dos pilares da programação, pois nos permitem organizar, simplificar e modularizar nosso código.

Por que usar funções?

- **Reutilização de Código:** Escreva uma vez, use quantas vezes precisar.
- **Organização:** Divida um problema complexo em partes menores e mais fáceis de gerenciar.
- **Legibilidade:** Um código bem organizado com funções é mais fácil de ler e entender.
- **Manutenção:** É mais simples corrigir ou atualizar um pequeno bloco de código isolado do que um script gigante.

1.1. Definindo e Chamando uma Função

Usamos a palavra-chave `def` para criar uma função.

A screenshot of a code editor with a dark background and light-colored text. The code is written in Python and consists of six lines. Line 1 is a comment: '# Definindo uma função simples'. Line 2 starts a function definition: 'def saudar():'. Line 3 is an indented print statement: 'print("Olá! Bem-vindo(a) ao mundo das funções em Python.")'. Line 4 is an empty line. Line 5 is a comment: '# Chamando (executando) a função'. Line 6 calls the function: 'saudar()'. The code is color-coded: comments are green, the function keyword 'def' is blue, and the function name 'saudar()' is orange.

```
1 # Definindo uma função simples
2 def saudar():
3     print("Olá! Bem-vindo(a) ao mundo das funções em Python.")
4
5 # Chamando (executando) a função
6 saudar()
```

1.2. Parâmetros e Argumentos

Podemos passar informações para dentro das funções através de **parâmetros**. Os valores que passamos ao chamar a função são chamados de **argumentos**.

```
1 def saudar_usuario(nome):
2     print(f"Olá, {nome}! Tudo bem?")
3
4 # Passando o argumento "Maria" para o parâmetro "nome"
5 saudar_usuario("Maria")
6 saudar_usuario("João")
7
```

1.3. O Comando **return**

Funções podem "devolver" um valor. Para isso, usamos o comando **return**.

```
1 def somar(a, b):
2     return a + b
3
4 resultado = somar(10, 5)
5 print(f"O resultado da soma é: {resultado}") # Saída: O resultado da soma é: 15
6 print(f"Outro resultado: {somar(100, 200)}") # Saída: Outro resultado: 300
```

1.4. Argumentos Padrão e Nomeados

Podemos definir um valor padrão para um parâmetro. Se nenhum argumento for passado, o valor padrão será usado.

```
1 def calcular_potencia(base, expoente=2):
2     return base ** expoente
3
4 print(calcular_potencia(5))      # Usa o expoente padrão 2. Saída: 25
5 print(calcular_potencia(5, 3))  # Passa o expoente 3. Saída: 125
```

1.5. Variáveis complexas

Em python é possível passar como parâmetro e retornar variáveis complexas sem a necessidade de ponteiros.

```
1 def soma_vetorial(vetor):
2     total = 0
3     for num in vetor:
4         total += num
5     return total
6
7 numeros = [1, 2, 3, 4, 5]
8 soma = soma_vetorial(numeros) # Saída: 15
```

Também é possível retornar múltiplas variáveis com o return.

```
1 def fibonacci(n):
2     a, b = 0, 1
3     v = []
4     for _ in range(n):
5         a, b = b, a + b
6         v.append(a)
7     return v
8
9 print(fibonacci(5)) # Saída: [1, 1, 2, 3, 5]
```

1.4. Argumentos Padrão e Nomeados

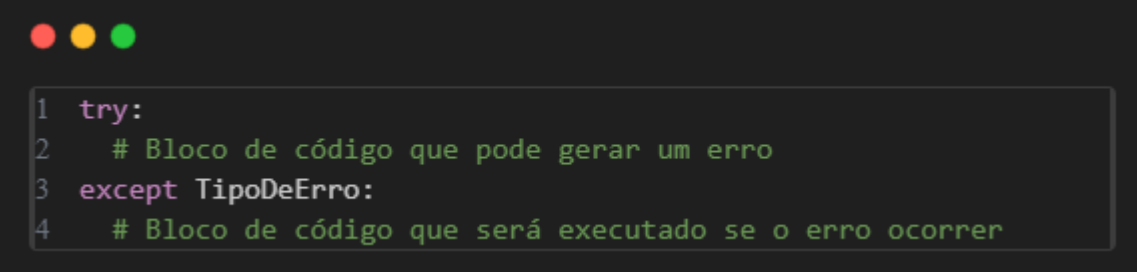
Podemos definir um valor padrão para um parâmetro. Se nenhum argumento for passado, o valor padrão será usado.

```
1 def calcular_potencia(base, expoente=2):
2     return base ** expoente
3
4 print(calcular_potencia(5))          # Usa o expoente padrão 2. Saída: 25
5 print(calcular_potencia(5, 3))      # Passa o expoente 3. Saída: 125
```

2. Lidando com Erros: `try` e `except`

Nem sempre o código funciona como esperamos. O usuário pode digitar um texto onde um número é esperado, ou podemos tentar dividir um número por zero. O bloco `try...except` nos permite "tentar" executar um código e "capturar" os erros que podem ocorrer, tratando-os de forma elegante sem que o programa quebre.

Estrutura básica:



```
1 try:
2     # Bloco de código que pode gerar um erro
3 except TipoDeErro:
4     # Bloco de código que será executado se o erro ocorrer
```

Exemplo prático:

```
1 def dividir(dividendo, divisor):
2     try:
3         resultado = dividendo / divisor
4         print(f"O resultado da divisão é {resultado}")
5     except ZeroDivisionError:
6         print("Erro: Não é possível dividir um número por zero!")
7     except TypeError:
8         print("Erro: Certifique-se de que ambos os valores são números.")
9
10 dividir(10, 2)
11 dividir(10, 0)
12 dividir("dez", 2)
```

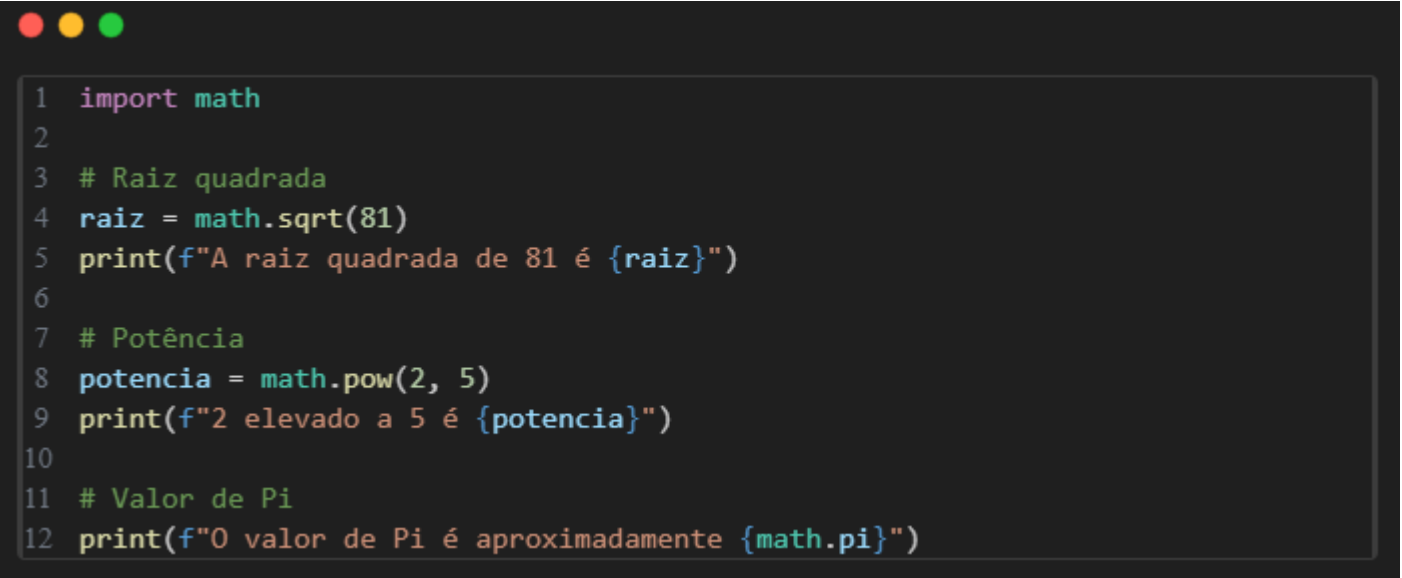
3. Expandindo Horizontes com Bibliotecas (Módulos)

Bibliotecas (ou módulos) são coleções de funções e ferramentas pré-escritas por outras pessoas que podemos "importar" para nossos projetos, evitando que tenhamos que reinventar a roda.

Usamos o comando `import` para trazer uma biblioteca para o nosso código.

3.1. Biblioteca `math`

Fornece acesso a funções matemáticas.

A screenshot of a code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Python and demonstrates the use of the 'math' module. It includes line numbers from 1 to 12 on the left side of the code block. The code imports the 'math' module, calculates the square root of 81, calculates 2 to the power of 5, and prints the value of Pi.

```
1 import math
2
3 # Raiz quadrada
4 raiz = math.sqrt(81)
5 print(f"A raiz quadrada de 81 é {raiz}")
6
7 # Potência
8 potencia = math.pow(2, 5)
9 print(f"2 elevado a 5 é {potencia}")
10
11 # Valor de Pi
12 print(f"O valor de Pi é aproximadamente {math.pi}")
```

3.2. Biblioteca `random`

Usada para gerar números aleatórios e fazer escolhas aleatórias.

```
1 import random
2
3 # Gera um número inteiro aleatório entre 1 e 100
4 numero_aleatorio = random.randint(1, 100)
5 print(f"Número sorteado: {numero_aleatorio}")
6
7 # Escolhe um item aleatório de uma lista
8 frutas = ["Maçã", "Banana", "Laranja", "Uva"]
9 fruta_escolhida = random.choice(frutas)
10 print(f"A fruta do dia é: {fruta_escolhida}")
```


3.3. Biblioteca `datetime`

Permite trabalhar com datas e horas de forma simples.

A screenshot of a code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Python and uses the `datetime` library. It includes line numbers from 1 to 9 on the left side of the code block. The code imports `datetime`, gets the current date and time, prints it, formats it as 'Dia/Mês/Ano', and prints the formatted string.

```
1 import datetime
2
3 # Pega a data e hora atuais
4 agora = datetime.datetime.now()
5 print(f>Data e hora atuais: {agora}")
6
7 # Formata a data para um formato mais legível (Dia/Mês/Ano)
8 data_formatada = agora.strftime("%d/%m/%Y")
9 print(f>Data formatada: {data_formatada}")
```